



Programmieren

Falk Morgenstern
Leon Johann Brettin

Foliensatz basierend auf Unterlagen von Prof. Huhn, Prof. Meyer und weiteren.
Nur zur Verwendung in der Vorlesung „Programmieren“ an der Ostfalia.
Keine Weitergabe.



Beispiel: Interface

```
public interface Animal {  
  
    boolean likes(String food);  
    void speak();  
    void getAngry();  
  
}
```



- Die Methoden eines Interface sind öffentlich und abstract (`public`, `abstract`)
- (letzteres gilt nicht für Default- und statische Methoden).
- Alle Attribute sind immer Konstanten (`public static final`).



Beispiel: Interface

- Damit eine Klasse das Interface `Animal` realisiert, muss es das Interface `Animal` implementieren, d.h. Implementierungen für alle Methoden von `Animal` bereitstellen.
- `Bird` implementiert `Animal`.
- Die Schnittstelle ist in diesem Beispiel `Animal`.
- `Bird` ist unsere Implementierung.

```
public class Bird implements Animal {  
  
    public boolean likes(String food) {  
        return food.equals("corn");  
    }  
  
    public void speak() {  
        System.out.println("beep");  
    }  
  
    public void getAngry() {  
        System.out.println("beep");  
    }  
  
}
```



Beispiel: Interface

```
IFlight[] flights = CSVReader.readAllLines("euro.csv");
```





Programmieren

4. UML

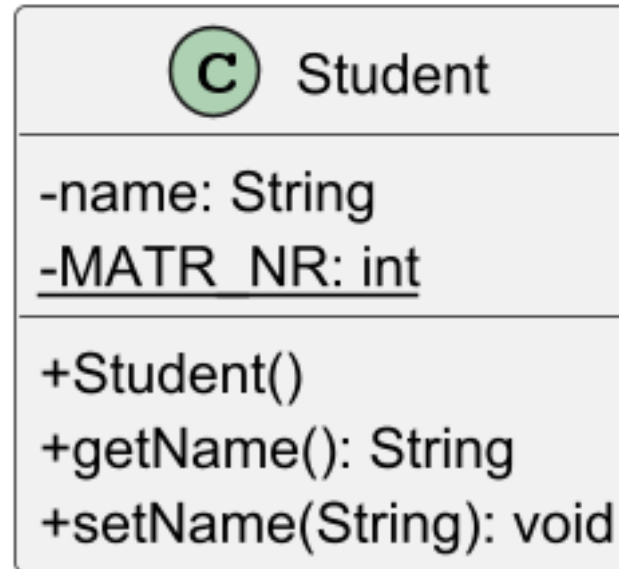


Unified Modeling Language

- Die UML (Unified Modeling Language) ist eine graphische Modellierungssprache zur
 - Spezifikation
 - Visualisierung
 - Dokumentation
 - Kommunikation
- über Software-Systeme.

Aufbau einer UML-Klasse

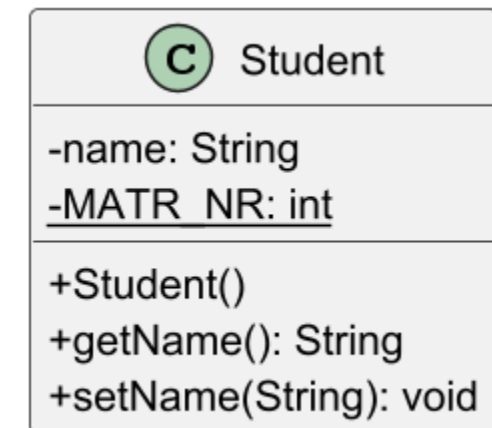
- Eine UML-Klasse besteht aus den folgenden drei Abschnitten:
 1. Name der Klasse
 2. Attributabschnitt
 3. Methodenabschnitt





Attributabschnitt

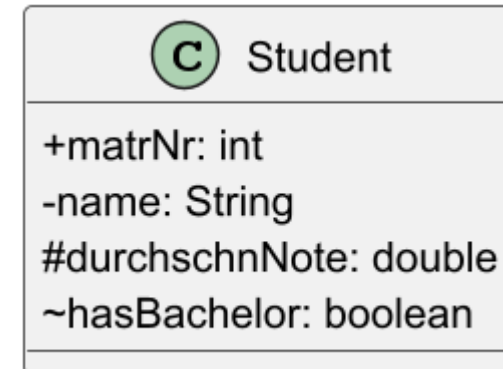
- Im Attributabschnitt werden alle Variablen, die in einer Klasse gespeichert sind dargestellt.
- Bei den Attributen steht immer zuerst ihre Sichtbarkeit, durch ein Zeichen dargestellt, dann der Name des Attributs und dann, mit einem Doppelpunkt getrennt, der Typ des Attributs.
- Bei den Attributen muss zwischen
 1. public
 2. private
 3. protected
 4. und paket-sichtbar
- unterschieden werden.
- public wird hierbei standardmäßig mit einem + dargestellt, private mit einem -, protected mit einem #, und paket-sichtbar mit ~





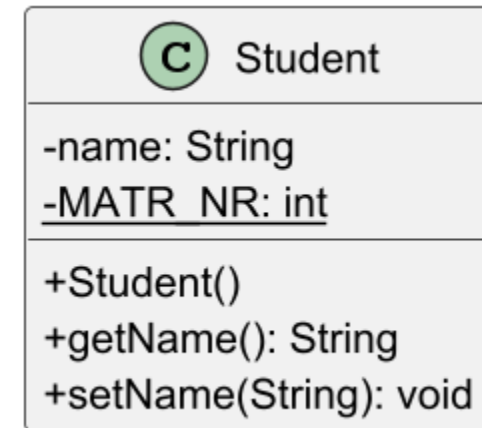
Beispiel: Attributabschnitt

```
public class Student {  
    public int matrNr;  
    private String name;  
    protected double durchschnNote;  
    boolean hasBachelor;  
}
```



Methodenabschnitt

- Im Methodenabschnitt werden alle Methoden einer Klasse definiert.
- Bei der Sichtbarkeit gelten die gleichen Bezeichner, wie bei den Attributen.
- Es wird
 1. erst die Sichtbarkeit angegeben,
 2. dann der Name der Methode,
 3. dann in Klammern der Typ der Parameter
 4. und dann, mit einem Doppelpunkt getrennt, der Rückgabewert der Methode.





Beispiel: Methodenabschnitt

```
public class Student {  
    public int matrNr;  
    private String name;  
    protected double durchschnNote;  
    boolean hasBachelor;  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public void setFirstAndLastName(String first,  
                                    String second) {  
        name = first + " " + second;  
    }  
}
```



C Student
+matrNr: int -name: String #durchschnNote: double ~hasBachelor: boolean
+Student() +setName(String): void +getName(): String +setFirstAndLastName(String, String): void



Programmieren

4. UML Modellierung

4.2 Beziehungen zwischen Klassen

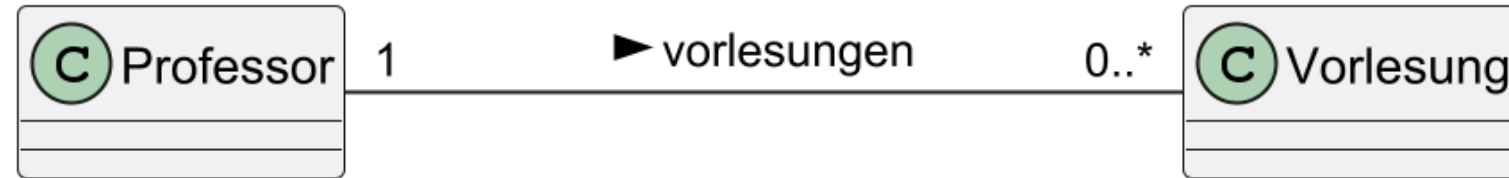


Beziehungen zwischen Klassen

- Zwischen zwei Klassen kann es unterschiedliche Arten von Beziehungen geben.
- Im allgemeinen kann man diese Beziehungen mit den vier Begriffen **Assoziation**, **Aggregation**, **Komposition**, **Vererbung** beschreiben

Assoziation und Aggregation

- Assoziation



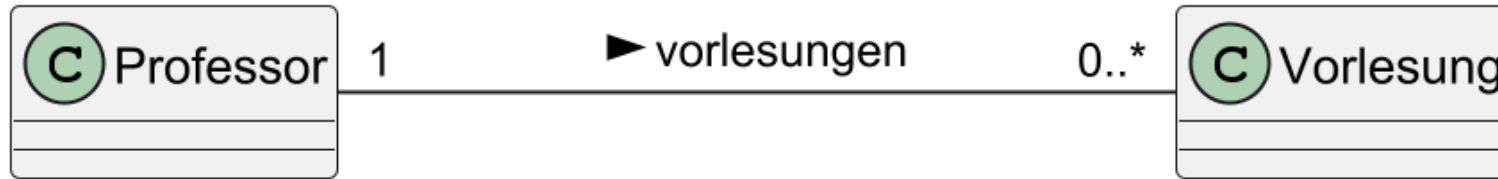
- Aggregation



- Komposition

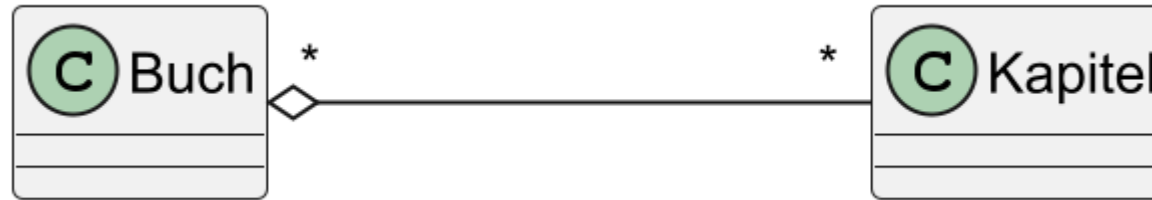


Assoziation



- Die Assoziation ist die allgemeinste Art der Beziehung.
- Die „1“ beim Professor steht für ein Professor-Objekt.
- Die „0..*“ steht für eine beliebige Anzahl oder 0.
- „vorlesungen“ ist der Name der Assoziation, das Dreieck bestimmt die Leserichtung.
- Jedes Professor-Objekt steht mit 0 bis beliebig vielen Vorlesungs-Objekten in Beziehung.

Aggregation



- Die Aggregation ist eine Art „Teil-Ganzes“ Relation.
- Die Lebensdauer der Teile hängt aber nicht von der Lebensdauer des ganzen ab.
- Man kann erst die einzelnen Kapitel schreiben, die dann irgendwann als Buch zusammengefasst werden können.

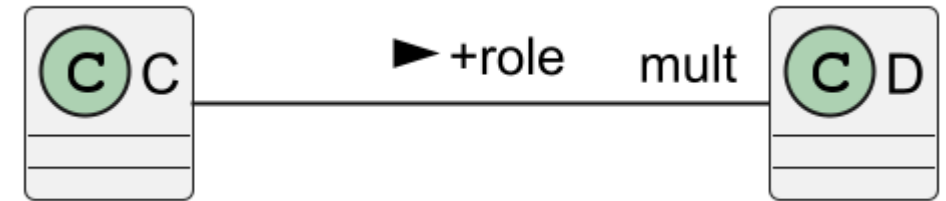
Komposition



- Teile können nur zu einem Ganzem gehören. Sie können nicht alleine existieren.
- Die Lebensdauer von Teilen ist auf die Lebensdauer von dem ganzen beschränkt.
- Ein Kunde kann 0 oder mehrere Konten auf der Bank haben. Jedes Konto kann aber nur zu einem Kunden gehören.
- Wenn der Kunde sein Verhältnis mit der Bank auflösen möchte, dann müssen auch seine Konten gelöscht werden.
- Manchmal auch starke Aggregation genannt, im folgenden nur Komposition

Java Implementierung von Assoziationen

- Assoziationen sind immer wie folgt aufgebaut:

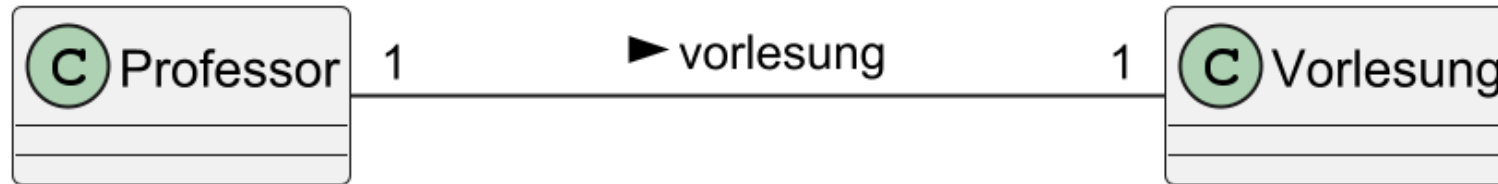


- Entscheidend für die Java-Implementierung sind *mult* und *role*.
 - role* steht für den Namen des Attributs.
 - mult* gibt die Multiplizität der Verbindung an, also wie viele Objekte jeweils in Beziehung stehen
 - Bei einer Multiplizität von 0, 1 oder 0..1 erhält man ein Attribut `D role`.
 - Bei einer Multiplizität > 1 oder $*$ wird ein Array oder eine andere Datenstruktur zum Speichern von mehreren Elementen benutzt.
 - Beispiel: `D [] role` oder `ArrayList<D> role`.



Beispiel: Java Implementierung von Assoziationen

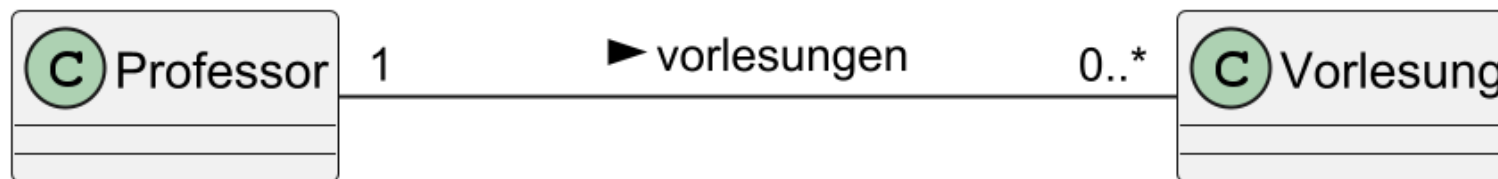
a)



```
public class Professor {  
    Vorlesung vorlesung;  
}
```



b)

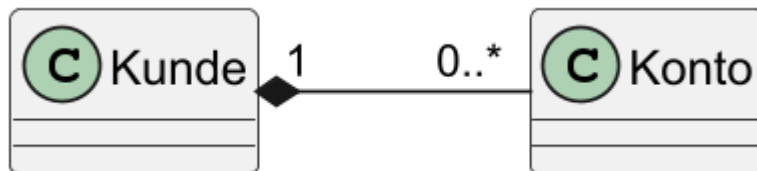


```
public class Professor {  
    Vorlesung[] vorlesungen;  
}
```



Beispiel: Implementierung einer Komposition

- Um eine starke Aggregation zu implementieren, müssen die Teile im Konstruktor erzeugt werden.
- Somit wird die Lebensdauer des Teils berücksichtigt
- Das Teil-Objekt soll keine eigene Beziehung zu äußeren Objekten haben.



```
public class Kunde {
    private LinkedList<Konto> konten;

    public Kunde() {
        konten = new LinkedList<Konto>();
    }
}
```



Beispiel: Implementierung einer Komposition

- Das Auto hat einen Motor
- Der Motor ist von der Existenz des Autos abhängig
 - Mit dem Auto wird auch der Motor erzeugt und vernichtet
- Von außen keine Referenz auf den Motor
- Klassendiagramm dazu?

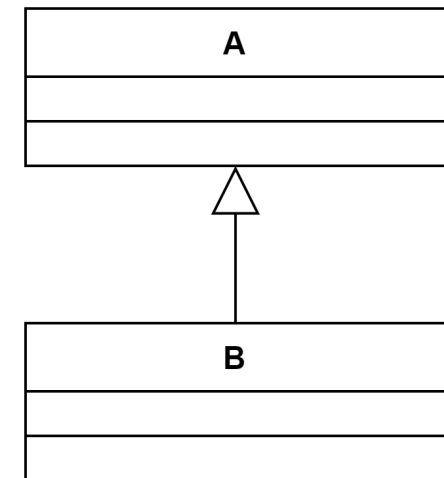
```
public class Car {  
    private Engine engine;  
  
    Car(EngineSpecs specs) {  
        engine = new Engine(specs);  
    }  
  
    void move() {  
        engine.work();  
    }  
}
```



Vererbung

- Klasse B erbt von Klasse A
- Klasse B ist Unterklasse, Subklasse oder abgeleitete Klasse von A
- Klasse A ist Oberklasse, Superklasse oder Basisklasse von B
- Vererbung: Pfeil mit durchgezogener Linie

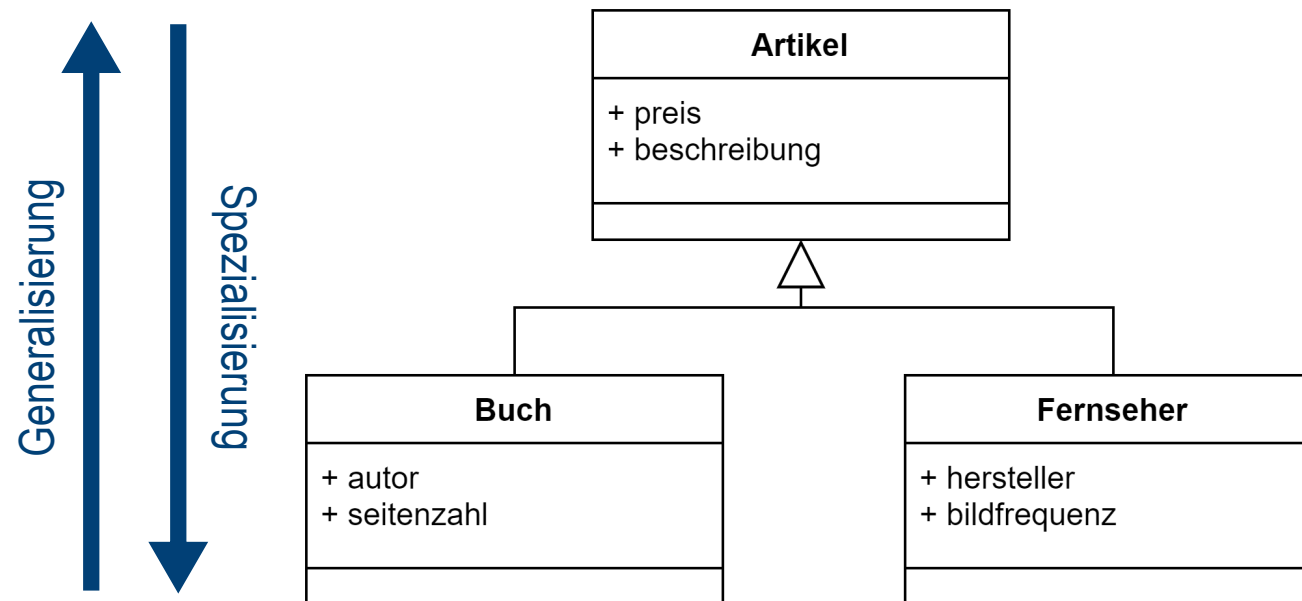
```
public class A {  
    // ...  
}  
  
public class B extends A {  
    // Vererbung in Java kennen wir aber noch  
    // nicht...  
}
```





Vererbung – Spezialisierung, Generalisierung

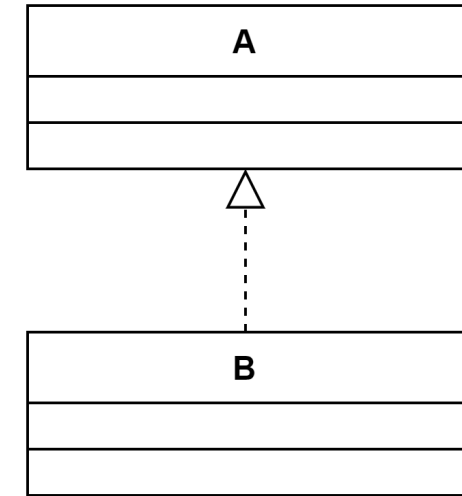
- Je nach Sichtweise ist Vererbung eine Spezialisierung oder Generalisierung





Realisierungsbeziehung / Interfaces

- Eine Klasse A gibt eine Spezifikation vor, die von einer Klasse B implementiert wird.
- Die Realisierungsbeziehung wird mit einer nicht ausgefüllten Pfeilspitze und gestrichelter Linie dargestellt.
- Meist wird die Realisierungsbeziehung in Verbindung mit Interfaces benutzt (Schnittstellenrealisierungsbeziehung)



```
public interface A {  
    // ...  
}  
  
public class B implements A {  
    // kennen wir auch noch nicht ...  
}
```





Programmieren

4. UML Modellierung

4.3 Erstellen von Klassendiagrammen in der Analyse-Phase

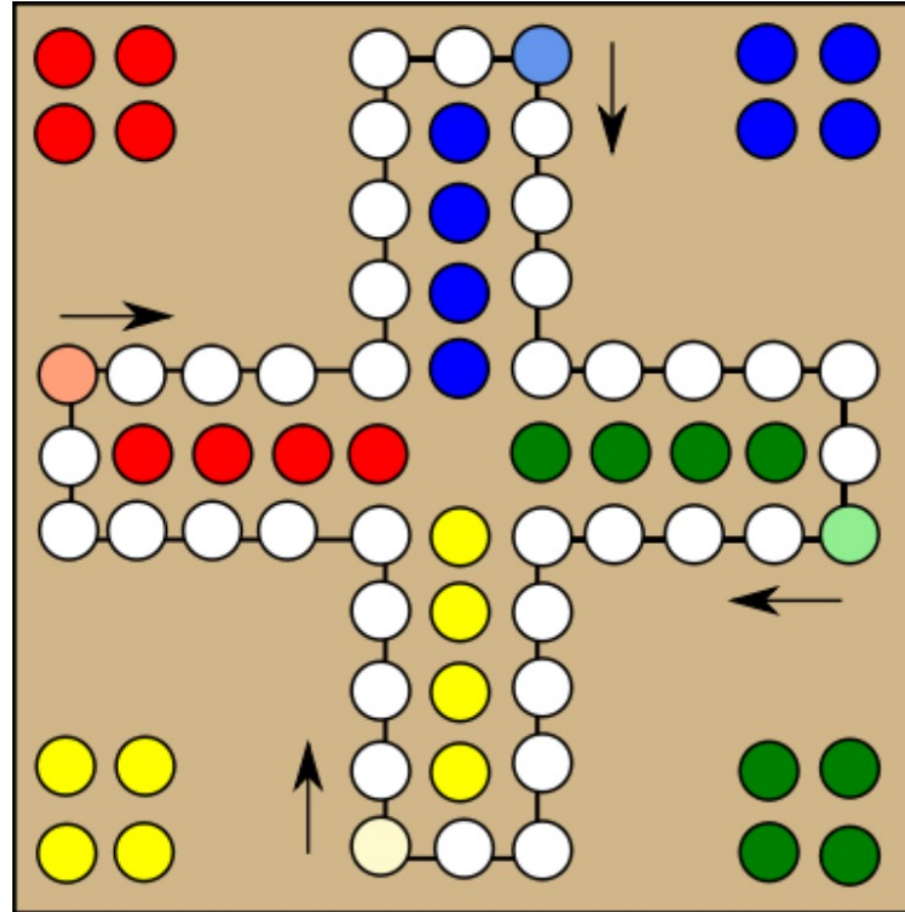


UML Klassendiagramme

- UML Klassendiagramme beschreiben die Struktur des Codes: Ergebnis der Analyse und Grundlage für Design und Implementierung -
- Klassendiagramm zeigt Entwicklersicht
 - Eine Klasse ist eine Schablone für Instanzen (Objekte), die zur Laufzeit existieren.
- Alle Elemente eines Klassendiagramms haben eine Entsprechung in Code
 - UML Attribute $\leftrightarrow$ Java-Attribute
 - UML-Methoden $\leftrightarrow$ Java- Methoden
 - Beziehungen $\leftrightarrow$ Java-Attribute (Methodenaufrufe)
 - UML-Vererbung $\leftrightarrow$ Java-Vererbung



Wie erstelle ich ein Klassendiagramm?



©Wikipedia

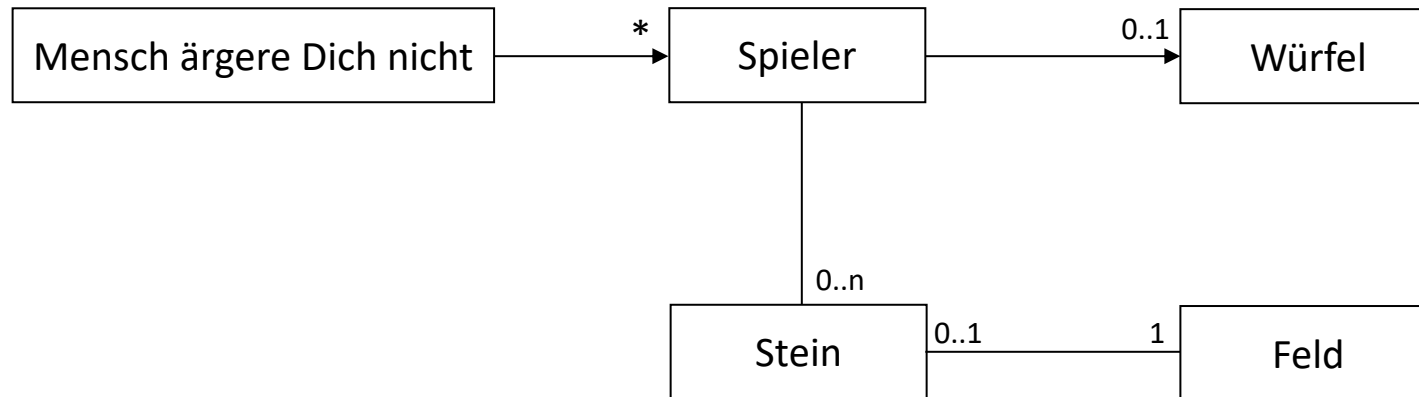
Begriffsmodell

- Sammlung wichtiger Begriffe (fachlich)
- Festlegung konkreter Namen für Objekte
- Beschreibung von Use Case Szenarien



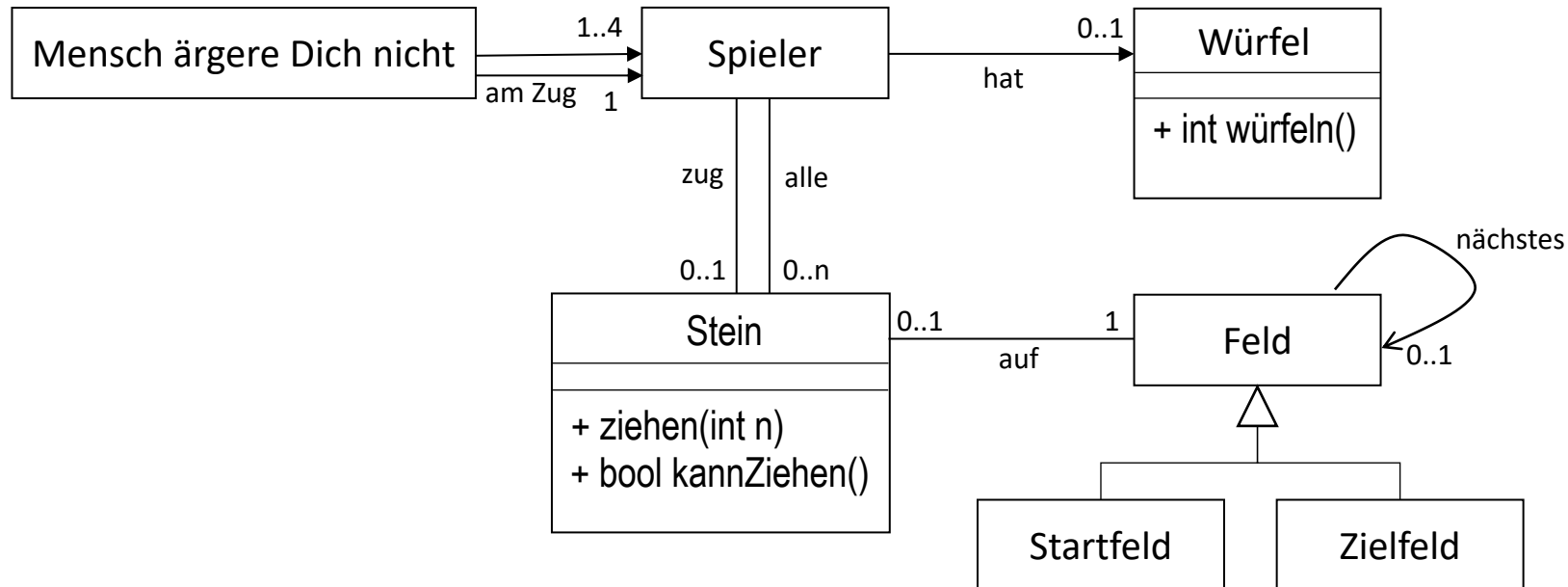
©Wikipedia

Begriffsmodell



©Wikipedia

Begriffsmodell – Verfeinerung



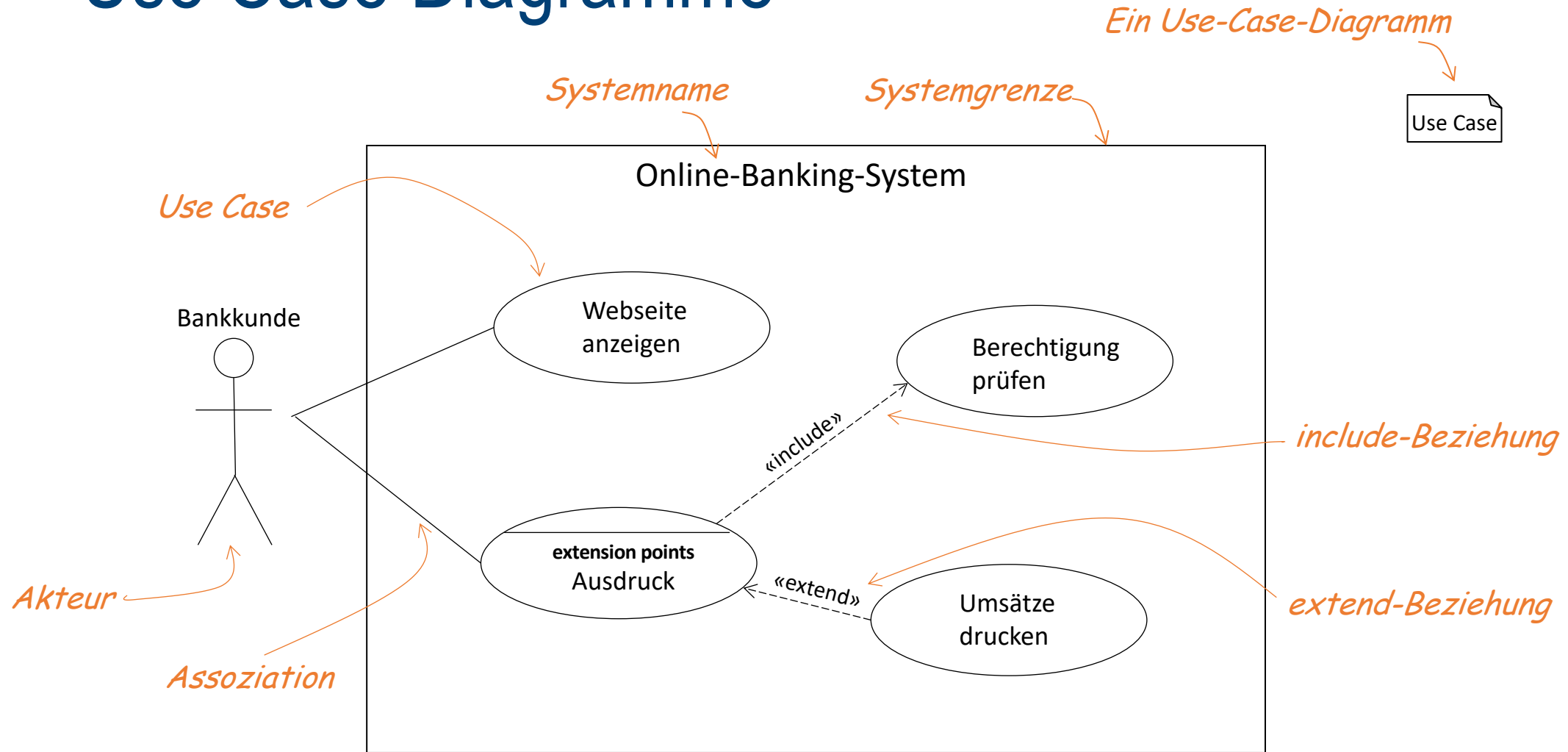
©Wikipedia



Use Case Diagramme

- Nützlich in Frühphasen eines Projekts zur Beantwortung der Frage „**Was** soll das geplante System leisten?“
- Zeigt das **externe Verhalten** aus Sicht des Nutzers („Akteurs“)
 - ... als grobe Funktionsbeschreibung
 - ... gegliedert in handhabbare logische Teile
 - ... zur Darstellung der Außenschnittstellen und Kommunikationspartner

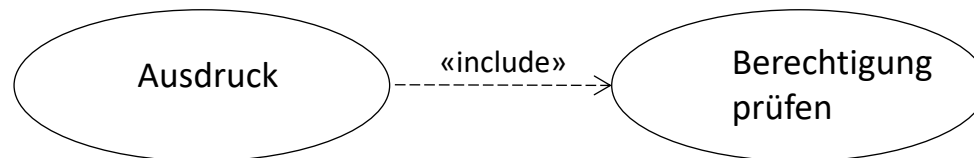
Use Case Diagramme



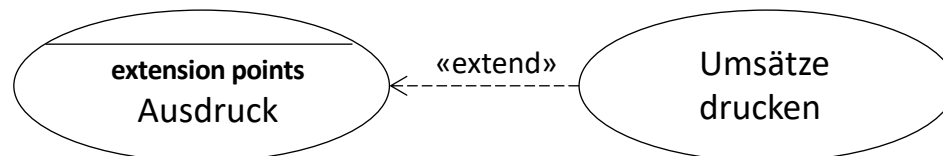
Use Case Diagramme

Sparsam verwenden, machen das Diagramm schwerer verständlich!

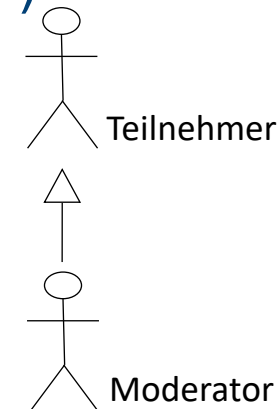
- Notationelle Freiheit
 - Akteur z.B. als „Strichmännchen“ oder auch als eignes Symbol   ...
- **Include**: „import“ eines weiteren use case - nicht optional, wird immer „ausgeführt“



- **Extend**: Mögliche (aber nicht verpflichtende, d.h. optionale) Erweiterung



- **Generalisierung von Akteuren**





Use Case Diagramme

- Zum Diagramm gehört eine textuelle/tabellarische Beschreibung für jeden Use Case:

Name:	
Kurzbeschreibung:	
Akteure:	
Vorbedingung:	
Fachlicher Auslöser:	
Normalablauf:	
Alternative Abläufe:	
Nachbedingung:	

Beispiel aus anderer Domäne

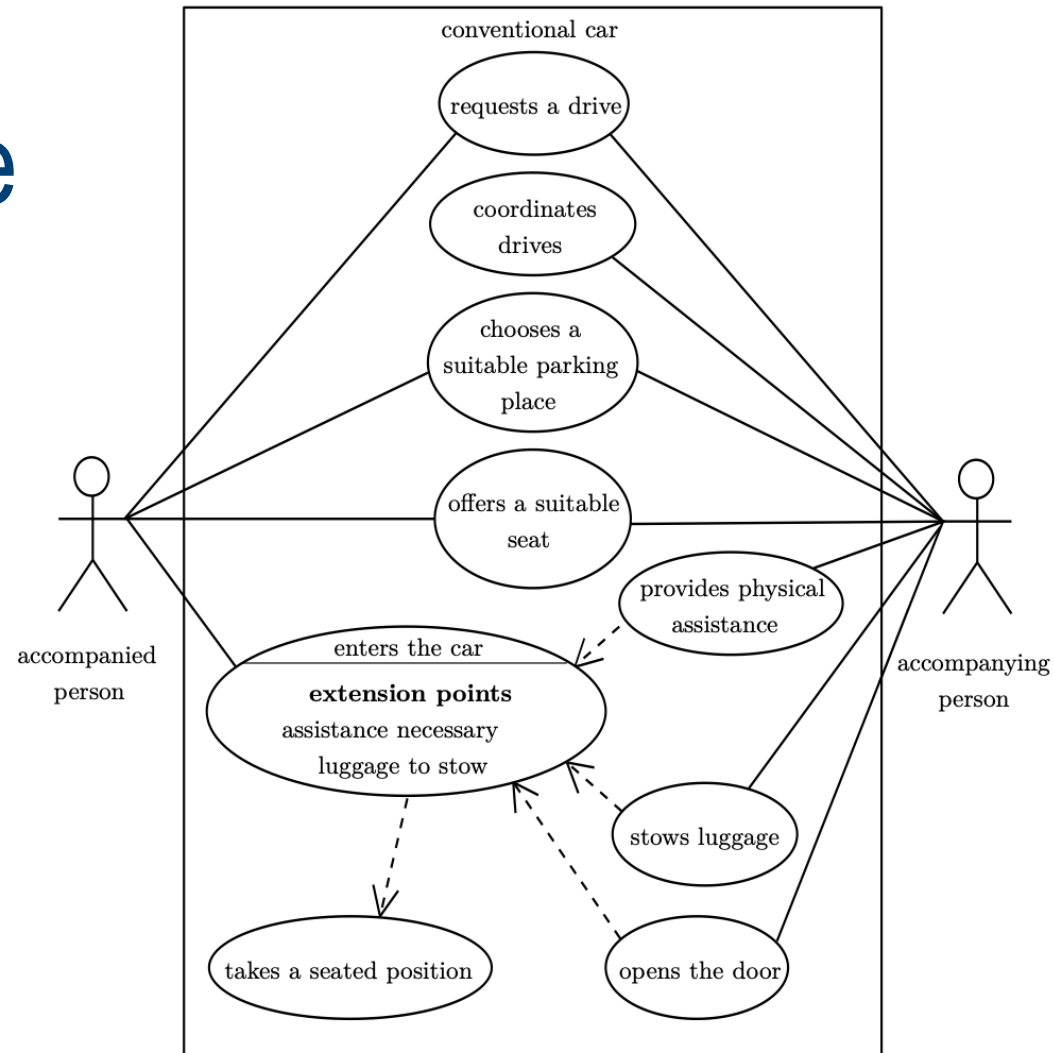


Fig. 1. Activities before and after an accompanied ride of an elderly person

Schrader, Tobias & Stolte, Torben & Jatzkowski, Inga & Graubohm, Robert & Maurer, Markus. (2019). An Approach for a Requirement Analysis for an Autonomous Family Vehicle. 1597-1603. 10.1109/IVS.2019.8813882.